

MODELAGEM E DESENVOLVIMENTO DE BANCO DE DADOS

SEMANA 17

Cláusulas **SELECT**: consultas, agregações, JOINS e subconsultas

Roteiro completo: teoria + atividade prática guiada

Carga horária: 3 aulas de 50 minutos

Ambiente da prática: OneCompiler — SQLite

Adaptação para o laboratório
Os computadores não dependem de MySQL instalado. Todo o roteiro prático foi convertido para SQLite e pode ser executado no OneCompiler. Comandos específicos do MySQL foram substituídos ou mantidos apenas como explicação conceitual quando necessário.

PARTE 1 — ROTEIRO DE AULA

Aula 1 — Teórica e demonstrativa | 50 minutos

1. Objetivos essenciais

- Revisar SELECT, WHERE, LIKE e ORDER BY.
- Compreender SUM, AVG e COUNT.
- Entender como JOIN combina dados de tabelas relacionadas.
- Diferenciar INNER JOIN e LEFT JOIN; reconhecer conceitualmente o RIGHT JOIN.
- Entender subconsultas em SELECT, WHERE e FROM.
- Aplicar esses recursos em relatórios simples de um e-commerce.

2. Organização das 3 aulas

Aula	Tempo	Foco
1	50 min	Teoria: SELECT, filtros, agregações, JOINS e subconsultas
2	50 min	Prática: criação do banco, SELECT, WHERE, ORDER BY, agregações e JOINS
3	50 min	Prática: subconsultas em SELECT, WHERE e FROM + evidências

3. Aula 1 — minuto a minuto

- 0–5 min — Situação: quais informações um gerente precisa em um relatório de vendas?
- 5–15 min — SELECT, WHERE, LIKE e ORDER BY.
- 15–25 min — SUM, AVG e COUNT.
- 25–35 min — INNER JOIN e LEFT JOIN.
- 35–47 min — Subconsultas em SELECT, WHERE e FROM.
- 47–50 min — Síntese para lousa e explicação da prática no OneCompiler.

4. SELECT básico

SELECT escolhe quais dados serão mostrados no resultado.

```
SELECT nome, email  
FROM clientes;
```

Para mostrar todas as colunas:

```
SELECT *  
FROM clientes;
```

Analogia

Imagine uma planilha com várias colunas. SELECT é a escolha de quais colunas você deseja enxergar no relatório.

5. WHERE, LIKE e ORDER BY

WHERE filtra registros.

```
SELECT nome, cidade  
FROM clientes  
WHERE cidade = 'São Paulo';
```

LIKE procura padrões em textos:

```
SELECT nome
FROM clientes
WHERE nome LIKE 'A%';
```

ORDER BY organiza o resultado:

```
SELECT nome, cidade
FROM clientes
ORDER BY nome ASC;
```

Para lembrar

WHERE = filtra. ORDER BY = ordena. LIKE = procura um padrão em texto.

6. Funções de agregação

Agregações resumem vários registros em um único valor ou em valores agrupados.

```
SELECT SUM(valor_total) AS total_vendas
FROM compras;
```

```
SELECT AVG(valor_total) AS media_compras
FROM compras;
```

```
SELECT COUNT(*) AS quantidade_compras
FROM compras;
```

Analogia

Em vez de ler compra por compra, SUM soma o conjunto, AVG encontra a média e COUNT conta quantos registros existem.

7. JOINS

JOIN combina dados de duas tabelas relacionadas por uma coluna em comum.

```
SELECT c.nome, c.email, p.valor_total
FROM clientes c
INNER JOIN compras p
    ON c.cliente_id = p.cliente_id;
```

INNER JOIN mostra somente registros que possuem correspondência nas duas tabelas.

```
SELECT c.nome, c.email, p.valor_total
FROM clientes c
LEFT JOIN compras p
    ON c.cliente_id = p.cliente_id;
```

LEFT JOIN mantém todos os clientes, mesmo os que ainda não possuem compras. Os dados da compra podem aparecer como NULL.

RIGHT JOIN

O material original também apresenta RIGHT JOIN. Explique o conceito, mas não o torne obrigatório na prática. Para turmas iniciantes, INNER JOIN e LEFT JOIN são suficientes para compreender o relacionamento entre tabelas.

8. Subconsultas

Uma subconsulta é uma consulta SQL colocada dentro de outra consulta.

Analogia

Consulta externa = pergunta principal. Subconsulta = uma pergunta auxiliar usada para descobrir um valor necessário para responder a pergunta principal.

9. Subconsulta em SELECT

```
SELECT c.nome,
      (
        SELECT SUM(p.valor_total)
        FROM compras p
        WHERE p.cliente_id = c.cliente_id
      ) AS total_compras
FROM clientes c;
```

A subconsulta soma o valor das compras de cada cliente e cria a coluna total_compras.

10. Subconsulta em WHERE

```
SELECT nome, email
FROM clientes
WHERE cliente_id IN (
  SELECT cliente_id
  FROM compras
  WHERE valor_total > (
    SELECT AVG(valor_total)
    FROM compras
  )
);
```

A parte mais interna calcula a média das compras. Depois, a consulta identifica compras acima dessa média e, por fim, retorna os clientes correspondentes.

11. Subconsulta em FROM

```
SELECT sub.nome, sub.total_compras
FROM (
  SELECT c.nome,
        SUM(p.valor_total) AS total_compras
  FROM clientes c
  JOIN compras p
  ON c.cliente_id = p.cliente_id
  GROUP BY c.cliente_id, c.nome
) AS sub
WHERE sub.total_compras > (
  SELECT AVG(valor_total)
  FROM compras
);
```

Nesse caso, a subconsulta de FROM cria um resultado intermediário que funciona como uma tabela temporária chamada sub.

12. MySQL x SQLite no OneCompiler

Nos materiais	No OneCompiler/SQLite	Motivo
CREATE DATABASE / USE	Não utilizar	O editor já executa no banco SQLite da sessão.
VARCHAR / DECIMAL	TEXT / REAL	Simplificação didática para SQLite.
MONTH() / YEAR()	strftime()	Funções de data são diferentes.
RIGHT JOIN	Apenas conceito	A prática usa INNER e LEFT para

13. O que colocar na lousa

SELECT = escolhe dados
WHERE = filtra
ORDER BY = ordena
LIKE = busca padrão de texto

SUM() = soma
AVG() = média
COUNT() = conta

INNER JOIN = somente correspondências
LEFT JOIN = mantém tudo da tabela da esquerda

SUBCONSULTA = consulta dentro de outra consulta
SELECT → cria informação adicional
WHERE → ajuda a filtrar
FROM → gera resultado intermediário

PARTE 2 — ATIVIDADE PRÁTICA GUIADA

Aulas 2 e 3 — 100 minutos | OneCompiler + SQLite

1. Identificação

Nome	Turma	Data
Arthur Saraiva de França	3B	19/8/2026

2. Situação-problema

Você está ajudando uma empresa de e-commerce a produzir relatórios sobre clientes e compras. Durante as duas aulas práticas, você usará o mesmo banco de dados para começar com consultas simples e chegar a subconsultas.

3. Como abrir o ambiente

- Abra o OneCompiler.
- Escolha o editor SQLite.
- Apague o código de exemplo que aparecer.
- Cole o script da Etapa 1.
- Clique em Run para executar.

Importante

Não selecione MySQL. O roteiro foi preparado e testado em SQLite.

AULA 2 — 50 MINUTOS

Etapa 1 — Criando o banco de e-commerce

Copie e execute o script completo:

Primeiro crie as tabelas:

```
PRAGMA foreign_keys = ON;

DROP TABLE IF EXISTS compras;
DROP TABLE IF EXISTS clientes;

CREATE TABLE clientes (
    cliente_id INTEGER PRIMARY KEY,
    nome TEXT NOT NULL,
    email TEXT NOT NULL,
    cidade TEXT NOT NULL
);

CREATE TABLE compras (
    compra_id INTEGER PRIMARY KEY,
    cliente_id INTEGER NOT NULL,
    valor_total REAL NOT NULL,
    data_compra TEXT NOT NULL,
    FOREIGN KEY (cliente_id) REFERENCES clientes(cliente_id)
);
```

Depois insira os dados:

```
INSERT INTO clientes (cliente_id, nome, email, cidade) VALUES
(1, 'Ana Silva', 'ana@email.com', 'São Paulo'),
(2, 'Bruno Lima', 'bruno@email.com', 'Osasco'),
(3, 'Carla Souza', 'carla@email.com', 'São Paulo'),
(4, 'Diego Rocha', 'diego@email.com', 'Barueri');

INSERT INTO compras (compra_id, cliente_id, valor_total, data_compra) VALUES
```

```
(1, 1, 150.00, '2026-08-05'),  
(2, 1, 300.00, '2026-08-08'),  
(3, 2, 90.00, '2026-08-10'),  
(4, 3, 500.00, '2026-08-12'),  
(5, 3, 200.00, '2026-08-15');
```

Depois da execução, teremos 4 clientes e 5 compras. Diego foi incluído sem compras para que o LEFT JOIN fique fácil de visualizar.

Etapa 2 — SELECT básico

```
SELECT nome, email  
FROM clientes;
```

Observe se aparecem os nomes e os e-mails dos quatro clientes.

Etapa 3 — Filtro com WHERE

Execute:

```
SELECT nome, cidade  
FROM clientes  
WHERE cidade = 'São Paulo';
```

Quais clientes foram exibidos?

Ana Silva e Carla Souza.

Etapa 4 — Ordenação

```
SELECT nome, cidade  
FROM clientes  
ORDER BY nome ASC;
```

O resultado ficou em qual tipo de ordem?

Ordem alfabética crescente pelo nome.

Etapa 5 — Agregações

Execute uma consulta por vez:

```
SELECT SUM(valor_total) AS total_vendas  
FROM compras;  
  
SELECT AVG(valor_total) AS media_compras  
FROM compras;  
  
SELECT COUNT(*) AS quantidade_compras  
FROM compras;
```

Registre os resultados:

Total das vendas: R\$ 1.240,00

Média das compras: R\$ 248,00

Quantidade de compras: 5

Etapa 6 — INNER JOIN

```
SELECT c.nome, c.email, p.valor_total  
FROM clientes c  
INNER JOIN compras p  
ON c.cliente_id = p.cliente_id  
ORDER BY p.compra_id;
```

Pergunta: Diego apareceu? Por quê?

Não. Porque Diego não possui nenhuma compra e o INNER JOIN mostra apenas registros que possuem correspondência nas duas tabelas.

Etapa 7 — LEFT JOIN

```
SELECT c.nome, c.email, p.valor_total
FROM clientes c
LEFT JOIN compras p
    ON c.cliente_id = p.cliente_id
ORDER BY c.cliente_id, p.compra_id;
```

Pergunta: o que aparece na coluna valor_total para um cliente sem compra?

Aparece NULL.

AULA 3 — 50 MINUTOS

Etapa 8 — Preparação para subconsultas

Antes dos desafios, execute a média geral das compras:

```
SELECT AVG(valor_total) AS media_geral
FROM compras;
```

Anote a média: R\$ 248,00

Etapa 9 — Desafio 1: subconsulta em SELECT

Objetivo: listar cada cliente e o valor total de suas compras.

Dica: a consulta externa deve ler clientes. Dentro do SELECT, use uma subconsulta com SUM(valor_total) filtrando pelo cliente atual.

Escreva sua consulta:

```
SELECT c.nome,
    (
        SELECT SUM(p.valor_total)
        FROM compras p
        WHERE p.cliente_id = c.cliente_id
    ) AS total_compras
FROM clientes c;
```

Etapa 10 — Desafio 2: subconsulta em WHERE

Objetivo: listar clientes que possuem alguma compra com valor acima da média de todas as compras.

Dica: primeiro descubra a média com AVG(). Depois use esse resultado em uma subconsulta para localizar os cliente_id correspondentes.

Escreva sua consulta:


```

SELECT nome, email
FROM clientes
WHERE cliente_id IN (
    SELECT cliente_id
    FROM compras
    WHERE valor_total > (
        SELECT AVG(valor_total)
        FROM compras
    )
);

```

Etapla 11 — Desafio 3: subconsulta em FROM

Objetivo: criar um relatório com o total comprado por cliente e mostrar apenas os totais acima da média geral das compras.

Dica: dentro do FROM, crie uma consulta que faça JOIN e SUM() com GROUP BY. Dê um apelido a esse resultado, como sub.

Escreva sua consulta:

```

SELECT sub.nome, sub.total_compras
FROM (
    SELECT c.nome,
           SUM(p.valor_total) AS total_compras
    FROM clientes c
    JOIN compras p
    ON c.cliente_id = p.cliente_id
    GROUP BY c.cliente_id, c.nome
) AS sub
WHERE sub.total_compras > (
    SELECT AVG(valor_total)
    FROM compras
);

```

Etapla 12 — Evidências

PRINT 1 — Resultado das funções SUM, AVG e COUNT

onecompiler.com/sqlite#draft-qjfn

OneCompiler Upgrade AI SQLite Run Save Login

queries.sql

```

6 CREATE TABLE clientes (
7     cliente_id INTEGER PRIMARY KEY,
8     nome TEXT NOT NULL,
9     email TEXT NOT NULL,
10    cidade TEXT NOT NULL
11 );
12
13 CREATE TABLE compras (
14     compra_id INTEGER PRIMARY KEY,
15     cliente_id INTEGER NOT NULL,
16     valor_total REAL NOT NULL,
17     data_compra TEXT NOT NULL,
18     FOREIGN KEY (cliente_id) REFERENCES clientes(cliente_id)
19 );
20
21 INSERT INTO clientes (cliente_id, nome, email, cidade) VALUES
22 (1, 'Ana Silva', 'ana@email.com', 'São Paulo'),
23 (2, 'Bruno Lima', 'bruno@email.com', 'Osasco'),
24 (3, 'Carla Souza', 'carla@email.com', 'São Paulo'),
25 (4, 'Diego Rocha', 'diego@email.com', 'Barueri');
26
27 INSERT INTO compras (compra_id, cliente_id, valor_total, data_compra) VALUES
28 (1, 1, 150.00, '2026-08-05'),
29 (2, 1, 300.00, '2026-08-08'),
30 (3, 2, 90.00, '2026-08-10'),
31 (4, 3, 500.00, '2026-08-12');

```

I/O AI Agent 24 ms | 4.7 MB

STDIN

Input for the program (Optional)

Output:

total_vendas
1240.0
media_compras
248.0
quantidade_compras
5

Ready

PRINT 2 — Resultado de um JOIN funcionando

onecompiler.com/sqlite#draft-qjfn

OneCompiler Upgrade AI SQLite Run Save Login

queries.sql

```

6 CREATE TABLE clientes (
7     cliente_id INTEGER PRIMARY KEY,
8     nome TEXT NOT NULL,
9     email TEXT NOT NULL,
10    cidade TEXT NOT NULL
11 );
12
13 CREATE TABLE compras (
14     compra_id INTEGER PRIMARY KEY,
15     cliente_id INTEGER NOT NULL,
16     valor_total REAL NOT NULL,
17     data_compra TEXT NOT NULL,
18     FOREIGN KEY (cliente_id) REFERENCES clientes(cliente_id)
19 );
20
21 INSERT INTO clientes (cliente_id, nome, email, cidade) VALUES
22 (1, 'Ana Silva', 'ana@email.com', 'São Paulo'),
23 (2, 'Bruno Lima', 'bruno@email.com', 'Osasco'),
24 (3, 'Carla Souza', 'carla@email.com', 'São Paulo'),
25 (4, 'Diego Rocha', 'diego@email.com', 'Barueri');
26
27 INSERT INTO compras (compra_id, cliente_id, valor_total, data_compra) VALUES
28 (1, 1, 150.00, '2026-08-05'),
29 (2, 1, 300.00, '2026-08-08'),
30 (3, 2, 90.00, '2026-08-10'),
31 (4, 3, 500.00, '2026-08-12');

```

I/O AI Agent 24 ms | 4.7 MB

STDIN

Input for the program (Optional)

Output:

nome	email	valor_total
Ana Silva	ana@email.com	150.0
Ana Silva	ana@email.com	300.0
Bruno Lima	bruno@email.com	90.0
Carla Souza	carla@email.com	500.0
Carla Souza	carla@email.com	200.0
Diego Rocha	diego@email.com	

nome	total_compras
Ana Silva	450.0
Bruno Lima	90.0
Carla Souza	700.0

Ready

PRINT 3 — Resultado de uma subconsulta funcionando

← → ↺ 🌐 oncompiler.com/sqlite#draft-qjfn

OneCompiler Upgrade AI SQLite Run

🔍 I/O AI Agent 24 ms | 4.7 MB

Save Login

queries.sql

```
6 CREATE TABLE clientes (  
7     cliente_id INTEGER PRIMARY KEY,  
8     nome TEXT NOT NULL,  
9     email TEXT NOT NULL,  
10    cidade TEXT NOT NULL  
11 );  
12  
13 CREATE TABLE compras (  
14     compra_id INTEGER PRIMARY KEY,  
15     cliente_id INTEGER NOT NULL,  
16     valor_total REAL NOT NULL,  
17     data_compra TEXT NOT NULL,  
18     FOREIGN KEY (cliente_id) REFERENCES clientes(cliente_id)  
19 );  
20  
21 INSERT INTO clientes (cliente_id, nome, email, cidade) VALUES  
22 (1, 'Ana Silva', 'ana@email.com', 'São Paulo'),  
23 (2, 'Bruno Lima', 'bruno@email.com', 'Osasco'),  
24 (3, 'Carla Souza', 'carla@email.com', 'São Paulo'),  
25 (4, 'Diego Rocha', 'diego@email.com', 'Barueri');  
26  
27 INSERT INTO compras (compra_id, cliente_id, valor_total, data_compra) VALUES  
28 (1, 1, 150.00, '2026-08-05'),  
29 (2, 1, 300.00, '2026-08-08'),  
30 (3, 2, 90.00, '2026-08-10'),  
31 (4, 3, 500.00, '2026-08-12');
```

🕒 ⚙️

STDIN

Input for the program (Optional)

Output:

Ana Silva	ana@email.com	300.0
Bruno Lima	bruno@email.com	90.0
Carla Souza	carla@email.com	500.0
Carla Souza	carla@email.com	200.0

nome	email	valor_total
Ana Silva	ana@email.com	150.0
Ana Silva	ana@email.com	300.0
Bruno Lima	bruno@email.com	90.0
Carla Souza	carla@email.com	500.0
Carla Souza	carla@email.com	200.0
Diego Rocha	diego@email.com	

Light Wiki SQLite

Etapa 13 — Perguntas finais

1. Qual é a função do WHERE em uma consulta?

Filtrar os registros de acordo com uma condição.

2. Para que serve AVG()?

Serve para calcular a média dos valores.

3. Qual é a diferença básica entre INNER JOIN e LEFT JOIN?

O INNER JOIN mostra apenas registros que possuem correspondência. O LEFT JOIN mantém todos os registros da tabela da esquerda, mesmo sem correspondência.

4. O que é uma subconsulta?

É uma consulta SQL dentro de outra consulta.

5. Qual foi a principal diferença percebida entre uma subconsulta em SELECT e uma subconsulta em FROM?

A subconsulta em SELECT cria uma informação adicional para cada registro. A subconsulta em FROM cria um resultado intermediário que funciona como uma tabela.

Checklist

- ☐ Abri o OneCompiler em SQLite.
- ☐ Criei as tabelas clientes e compras.
- ☐ Executei SELECT e WHERE.
- ☐ Usei ORDER BY.
- ☐ Executei SUM, AVG e COUNT.
- ☐ Executei INNER JOIN.
- ☐ Executei LEFT JOIN.
- ☐ Realizei o desafio de subconsulta em SELECT.
- ☐ Realizei o desafio de subconsulta em WHERE.
- ☐ Realizei o desafio de subconsulta em FROM.
- ☐ Registre as evidências.
- ☐ Respondi às perguntas finais.

Critérios de avaliação — 10 pontos

Critério	Valor
----------	-------

Criação e execução da base de dados	1,0
SELECT, WHERE e ORDER BY	1,5
SUM, AVG e COUNT	1,5
JOINS	2,0
Subconsultas	2,5
Evidências	0,5
Perguntas e organização	1,0

PARTE 3 — ORIENTAÇÕES AO PROFESSOR

1. Estratégia para uma turma iniciante

Não avance para subconsultas enquanto a maior parte da turma ainda estiver com dificuldade em interpretar SELECT, FROM e WHERE. É melhor concluir menos comandos com compreensão do que copiar muitas consultas sem entender o fluxo.

2. Erros comuns

Esquecer ponto e vírgula

Mostre que o ponto e vírgula encerra a instrução SQL no roteiro adotado.

Usar texto sem aspas

Em filtros como cidade = 'São Paulo', o texto deve estar entre aspas simples.

Esquecer a condição do JOIN

Reforce que ON indica como as tabelas se relacionam.

Confundir SUM com COUNT

SUM soma valores; COUNT conta registros.

Tentar usar comandos do MySQL

Reforce que a prática é SQLite. Não usar CREATE DATABASE, USE, MONTH() ou YEAR().

Copiar a subconsulta inteira sem entender

Peça ao aluno para executar primeiro a consulta interna sozinha; depois encaixá-la na consulta externa.

3. Estratégia para ensinar subconsultas

Trabalhe de dentro para fora

Antes de mostrar uma consulta aninhada completa, execute a consulta interna separadamente. Mostre o resultado. Em seguida, coloque essa consulta dentro da consulta externa. Isso ajuda o estudante a enxergar o caminho do dado.

4. Validação técnica do roteiro

Os scripts desta atividade foram executados em SQLite durante a preparação do material. Foram validados: CREATE TABLE, INSERT, SELECT, WHERE, LIKE, ORDER BY, SUM, AVG, COUNT, INNER JOIN, LEFT JOIN, strftime() e as três subconsultas propostas.